Architecture Justification – Automated Rubric Assignment Evaluator (Submission Intake & Automated Evaluation Pipeline)

Name : Rushieswar Reddy **SRN :** PES1UG24AM158 **Section :** 5C

Architecture Selection: We chose Microservices Architecture for the Automated Rubric Assignment Evaluator System.

Architectural Choice: Microservices, with a Web Portal (Student and Faculty Evaluator) reaching the backend only through an API Gateway (ISecureAPI). Six small services are split along the requirements: Submission Intake (FR-001, FR-005), Evaluation Queue (FR-002), Sandbox Runner (FR-003), Rubric Scoring & Results (FR-004), Status & Notification (FR-005) and Assignment Config (manifest, rubric, test suite). Each service owns its data (for example the Submission Store and Results Store) and talks to the others only through interfaces such as IEnqueue, IRunJob, IArtifactFetch, ITestSuite and IScoreRequest, using gRPC, REST or asynchronous AMQP events.

Reason 1 – Untrusted code needs fault isolation (FR-003, NFR-002): The system compiles and executes student code by design, so the runner is the main failure surface. NFR-002 requires that a runner crash must not affect other queued submissions. As an independently deployed Sandbox Runner Service, a crash, infinite loop or killed sandbox only fails that run. The Evaluation Queue Service sees the failed acknowledgement, retries at most twice and then marks the run FAILED (FR-002), while all other submissions continue. A layered or single-server design would run the evaluator in the same process space as intake, scoring and status, so one bad submission could stall the whole queue.

Reason 2 – Deadline peak load needs independent scaling (NFR-001): An entire cohort submits in the final hour, and the system must take 100 concurrent submissions with zero dropped items and p95 latency of 10 s or less. Evaluation is the heavy, bursty part, whereas config, scoring and status are light. With microservices we run several Sandbox Runner instances and scale only that service (and Intake) during the peak, while the rest stay small. Client-Server would put everything on one server, which the handout lists as a single point of failure and scalability bottleneck, and Layered scales the whole application as one unit.

Security Advantage: Because the runner is its own service, it can be locked down to exactly what NFR-002 asks: network-isolated, read-only test-harness mount, 2-core CPU limit and 60 s timeout. It holds no database credentials and has no interface to the Submission Store or Results Store. It can only fetch the archive (IArtifactFetch) and the test suite (ITestSuite), and return outcomes (IScoreRequest). Even if student code escaped its sandbox, it would have no path to rubric marks or other students' submissions. The API Gateway is the single entry point: it authenticates every request, checks the role and ownership, and terminates HTTPS, so a Student only sees their own submission, status and breakdown.

Performance Benefit: The Evaluation Queue Service buffers submissions in a FIFO queue and dispatches them asynchronously to runners (IRunJob) as capacity frees. Submission Intake therefore returns the submission ID, SHA-256 hash and timestamp immediately and never blocks on test execution, and status updates reach the Student through asynchronous events (IStatusUpdate). Each service has its own resource limits, so runner instances can be capped to keep the total memory footprint within the 1.5 GB budget, while the queue prevents items from being dropped under load.

Trade-off: Microservices add operational complexity, network latency and distributed data-consistency issues. We limit this by using only six services that match the requirement boundaries, keeping all submission versions in one Submission Store (FR-005), and having the Scoring Service write the execution report and itemised breakdown to the Results Store together, so the total always equals the sum of the criterion marks (FR-004).